

Repair: Edit Programs, Not Rewrites

Individual component of FaithGuard · M.S.A Ahamed · Revised plan, 8 October 2026

This document describes one component of FaithGuard, a layer that checks AI answers about financial reports against the evidence they were given, and then sends each answer, fixes it, or withholds it with a reason. This component performs the repair: a small local model writes a checkable edit program that copies or calculates the correct numbers from table cells, an exact executor runs it, and a hard gate checks the result before it can reach the user.

Nothing in this document has been built or measured yet. Every result below is a hypothesis with a stated way to reject it. All example numbers are illustrative, not results.

1 Purpose

This component tests whether a small local model that writes **checkable edit programs** fixes wrong financial numbers while creating fewer new errors than free rewriting.

Working title: Edit Programs, Not Rewrites: Verifiable Repair of Numeric Errors in Financial Answers with Small Local Models.

When the decision policy chooses repair, the component:

1. writes an edit program over the flagged spans, using only four operations;
2. runs it with exact Decimal arithmetic over table cells;
3. recomputes or flags every claim that depended on a changed number;
4. checks the result with a hard gate, retries once with the failure reason, and abstains if it fails again.

It also owns the shared **calculation API and deterministic executor** that the rest of the system uses.

Example

The question is "What was the Group's profit after tax in FY2025?" The AI answers: "Group profit after tax rose 7.0% to Rs. 12,450 million." Detection has flagged the number (Bank figure, wrong scale) and the dependent "7.0%", and the policy has chosen repair.

Repair writes a program: COPY the Group 2025 cell, then CALCULATE growth from the Group 2025 and 2024 cells. The executor runs it, and the gate confirms the cell, period and unit, and that nothing else changed. The fixed answer is sent. Had the Group figure been missing, repair would return CANNOT_FIX, and the user would get the evidence and a reason instead.

```
COPY      "Rs. 12,450 million" ← cell [Group, FY2025]
CALCULATE "7.0%"              ← growth( cell [Group, FY2025], cell [Group, FY2024] )
```

The program above is illustrative; the exact syntax is fixed by the Pydantic schema.

2 Position in the system

Content	
Receives	For every answer the policy sends to repair: the answer, the flagged spans (gold spans, or predicted spans from Channel A or released LettuceDetect) and the frozen evidence.
Produces	The edit program, the executed repaired answer and the gate result, or CANNOT_FIX with a reason code. Repair outcomes are stored for the policy's replay study.
Does not do	It does not decide whether to repair (the decision policy does), does not detect spans (the detection component does), and does not repair non-numeric errors, which lead to abstention.

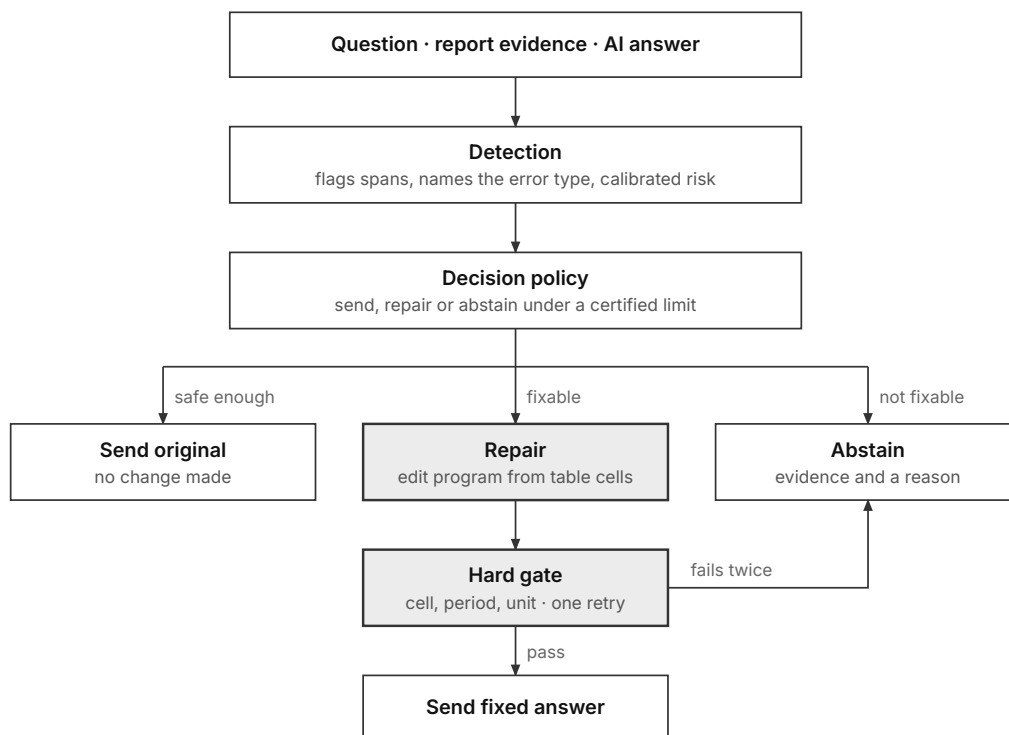


Figure 1. Where the component sits in FaithGuard. The shaded boxes are this component.

3 Research questions

1. At the same correction rate, do edit programs introduce fewer new numeric errors than free-text editing with the same model and data?
2. Does executor-verified self-training improve on supervised fine-tuning alone, especially on held-out error types and the held-out generator?
3. How much does repair degrade when spans come from a real detector, and does training on detector false positives (KEEP targets) close that gap?

4 Component architecture

The model writes a program; everything after it is deterministic. Training runs offline.

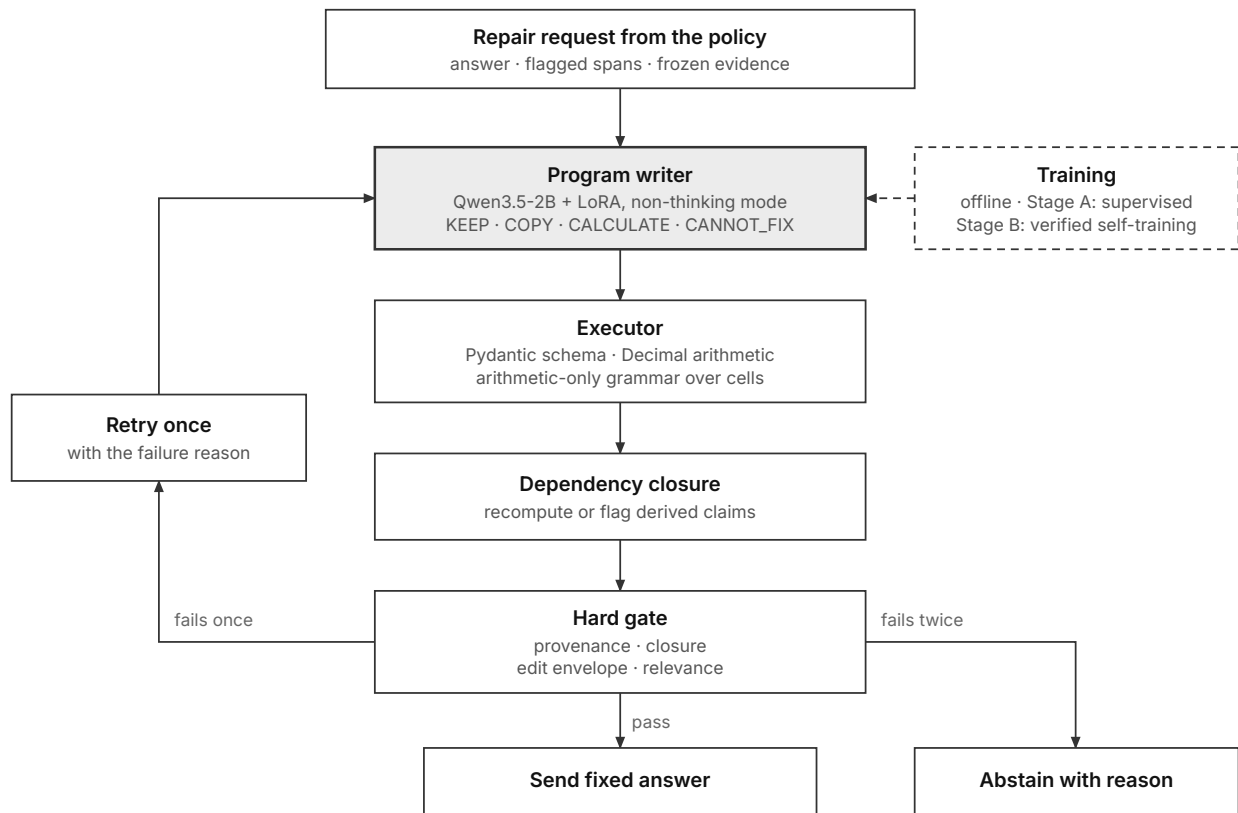


Figure 2. Internal architecture. The program writer (shaded) is the model this component trains. The dashed box runs offline.

Edit language

Operation	What it does
KEEP	Leaves a span unchanged
COPY	Copies a value from a table cell
CALCULATE	Computes a value from cells, with arithmetic only
CANNOT_FIX	Gives up with a reason code

Non-numeric errors lead to abstention. Free-text REPLACE moves to future work.

Executor

A Pydantic schema, Decimal arithmetic and a grammar that allows only arithmetic over cell references. Arbitrary code cannot be expressed.

Dependency closure

When a number changes, every claim derived from it (growth rates, comparisons, direction words) is recomputed or flagged.

Hard gate

Checks provenance (right cell, period, unit, scope), closure, the edit envelope (nothing outside the flagged spans changed) and relevance (the answer still answers the question). One retry with the failure reason, then abstain.

Model

Qwen3.5-2B with LoRA in non-thinking mode; 4B as a stretch. An fp16 pilot on a free T4 runs in week 1. If it fails, these are tried in order, all on a free T4:

1. QLoRA: the base model loaded in 4-bit, with the LoRA weights and maths in fp32 (slower, but avoids fp16 overflow);
2. a smaller Qwen3 model (1.7B, then 0.6B);
3. a prompt-only repairer, as described in Section 8.

Stage A · Supervised fine-tuning

Targets from FinQA and TAT-QA programs, XBRL substitutions, KEEP items from out-of-fold detector false positives, and CANNOT_FIX items with evidence deliberately removed.

Stage B · Executor-verified self-training

Sample several programs per item, keep only those the executor and gate accept, fine-tune on them, and repeat 2–3 rounds. It runs on a free T4. It is reported as rejection-sampling fine-tuning, not as RL. GRPO moves to future work: it might fit a free T4 with QLoRA, but it is slower and less stable, and no research question needs it.

5 Data and shared foundation

All three components use one benchmark, built once and frozen, so their results can be compared and combined.

Source	Role	Labels	Licence
RAGTruth	General span training for detection	Human spans	MIT
FinQA	Gold programs for repair targets; injected errors	Gold answers and programs	MIT
TAT-QA	Table-and-text arithmetic targets	Gold answers	CC BY 4.0
US XBRL facts (SEC EDGAR)	Automatic wrong-context negatives; operand checks	Automatic	Public filings
Team benchmark, US and Sri Lanka	Main natural-error test, calibration set, policy outcomes	Human, blinded	Team release terms

FinRank (fresh 2024–25 US questions, CC BY-NC 4.0) is the one optional external test.

Answers and evidence

- **Two generators:** Qwen3.5-9B and Gemma 4 12B-it, both Apache-2.0, run once at 4-bit with llama.cpp on a free T4 and stored. Two vendors are enough for a leave-one-generator-out test.
- **Questions:** 150 US and 250 Sri Lankan question groups from about 12 held-out issuers per country. Each question is written with its gold answer, gold table cells and question type, so code can check numbers later.
- **Evidence:** the main test uses hand-corrected tables, so table-reading errors cannot be mistaken for model errors. One automatic-extraction run, using the best of three tools on 20 tables, is reported as the real-world condition.
- **Retrieval is frozen.** Every method sees the same evidence for the same question.

Rules that keep results honest

- Training, development, calibration and test splits are issuer-disjoint: one company never appears in two splits.
- The test set is locked before any tuning. Hypotheses, the risk target ($\alpha = 0.10$), splits and primary comparisons are pre-registered before the test set is opened.
- One versioned record format is shared by all components. Gold labels live in a separate store that no component reads at decision time.
- Intervals use a bootstrap clustered by issuer. Counts of issuers, questions and answers are always reported separately.
- LLM judges are used for silver training labels only, never for test labels.

Shared tasks owned by this member

- The calculation API and the deterministic executor.
- With the other two members: Sri Lankan report collection and table correction (split by sector), and writing about 133 question groups with gold answers.

6 Experiments

Comparison	Question it answers
Free-text rewrite, same backbone and data (FRED-style)	RQ1, the main baseline
Rule-only COPY and CALCULATE	What rules alone can fix
Zero-shot programs from the untrained 2B model and from Qwen3.5-9B	Is training needed at all?
Supervised only vs supervised plus self-training	RQ2
Gold spans vs predicted spans; with vs without KEEP training	RQ3
Dependency closure on vs off	Does closure prevent half-fixed answers?

Metrics: correction rate, new-error rate, damage rate (correct claims made wrong), information preserved (human-scored sample), abstention calibration, valid-program rate in the deployed 4-bit runtime, and latency.

7 Hypotheses

Hypothesis	Confirmed if	Rejected if
H1. Programs create fewer new errors than free rewriting	Lower new-error rate at a matched correction rate	No difference; reported as a negative result
H2. Self-training improves out-of-distribution repair	Gains on held-out error types and generator	Gains only on seen error types
H3. KEEP training cuts harm from detector false positives	Lower damage rate with predicted spans	No change

8 Scope and compute

- **Minimum publishable result:** the executor, the supervised model, the free-rewrite baseline and the correction-vs-harm frontier on natural errors with gold spans.
- **If no fine-tuning works on a free T4:** the paper compares prompted edit programs with prompted free rewriting, using the same model and the same few examples. RQ1, the main question, still stands; RQ2 is reported as not tested, with the reason.
- **Stretch (all fit a free T4):** a Qwen3.5-4B backbone with QLoRA; Granite 4.2 3B replication; REPLACE with a neural checker.
- **Future work:** GRPO. It might fit a free T4 with QLoRA, but it needs more time and tuning than the schedule allows.
- **Stands alone:** it works with gold spans. Predicted spans come from Channel A, or from released LettuceDetect if Channel A is late.
- **Hands over:** repairer v0 (rule-only) by 30 November 2026 and repairer v1 (trained) frozen by 15 February 2027, for the policy’s transfer study.

Work on this member’s quota	Rough GPU hours
Repair fine-tuning: main model, free-rewrite baseline, KEEP ablation (Kaggle T4)	10–20
Repair self-training: 2–3 rounds of sampling and training	12–20
Share of pilots, debugging and reruns (15–25 hours across the team)	—

These figures are rough guesses, not measurements. The week-1 pilots replace them with real timings. The 4-bit repairer’s latency checks run on a laptop CPU.

9 Relation to existing work

Defensible claim: a model of at most 4B parameters writes typed, evidence-bound edit programs with bounded dependency closure. It is trained with supervised fine-tuning, then with executor-verified self-training. It is judged on correction against introduced harm, on natural errors, with both gold and predicted spans.

Claims to avoid: that executable edits, deterministic copying or a verify-repair-abstain loop are new in themselves. FRED, VeriFin and FinGround already cover parts of this.

Realistic first target: FinNLP 2027 or SLIIT ICAC 2027.

10 Risks and cut list

Risk	Early warning	Fallback
fp16 LoRA fails on a T4	Week-1 pilot loss diverges	QLoRA with fp32 maths; then a smaller Qwen3 model; then a prompt-only repairer
Generators make very few errors	Pilot error rate below about 10%	Report it as a finding that bounds the benefit; add harder question types
A teammate's part is late	A checkpoint is missed	Use gold spans, or released LettuceDetect for predicted spans
A new paper overlaps the contribution	Monthly literature refresh	Narrow the claim and cite it; never hide it
Free GPU quota is cut or runs out	Hours used pass the budget	Use Sharaf's spare quota; switch between Kaggle and Colab; apply the cut list

Cut list item for this component, in team order: the second and third self-training rounds (3rd).

Never cut: the human-labelled natural test set and the main comparison with free rewriting.

11 Plan

By	Milestone
18 Oct 2026	fp16 and QLoRA pilots done on a free T4; GPU quota recorded
31 Oct 2026	Thin end-to-end demo on 20–40 items, rules only
19 Oct – 30 Nov 2026	Executor, gate and rule-only repairer v0
30 Nov 2026	Repairer v0 handed over to the policy
1 Dec 2026 – 15 Jan 2027	Supervised fine-tuning
16 Jan – 15 Feb 2027	Self-training
15 Feb 2027	Repairer v1 frozen
16 Feb – 31 Mar 2027	Predicted-span tests
31 May 2027	Paper draft
30 Jun 2027	Final report and demo

References

1. FRED: Financial Retrieval-Enhanced Detection and Editing of Hallucinations in Language Models. arXiv:2507.20930, 2025.
2. FinGround: Detecting and Grounding Financial Hallucinations via Atomic Claim Verification. arXiv:2604.23588, 2026.
3. Z. Chen et al. FinQA (EMNLP 2021); F. Zhu et al. TAT-QA (ACL 2021).
4. T. Dettmers et al. QLoRA: Efficient Finetuning of Quantized LLMs. arXiv:2305.14314, 2023.

VeriFin is named in the text; its full citation is added once the source has been opened and logged in the team's related-work notes.